

3_Estruturas de Repetição (for ... in)

June 9, 2026

Imagine que você queira repetir um comando 3, 10 ou até 1.000 vezes. Escrever cada linha manualmente seria um desperdício de tempo e tornaria seu código difícil de manter. Veja abaixo o que deveríamos fazer para repetir um print 3x.

```
[2]: print("0 ifba é nota 7.0!")  
      print("0 ifba é nota 7.0!")  
      print("0 ifba é nota 7.0!")
```

```
0 ifba é nota 7.0!  
0 ifba é nota 7.0!  
0 ifba é nota 7.0!
```

Em vez de fazermos de forma manual, podemos um loop for, que automatiza repetições como o código abaixo. A variável lista_numeros tem uma lista de números que iremos iterar. Para cada um desses números iremos imprimir a mensagem do ifba. Na prática é o mesmo programa que o anterior porém mais eficaz pois caso queiramos alterar o texto, fazemos isso em apenas um lugar.

```
[3]: lista_numeros = [1 , 2 , 3]  
      for numero in lista_numeros:  
          print("0 ifba é nota 7.0!")
```

```
0 ifba é nota 7.0!  
0 ifba é nota 7.0!  
0 ifba é nota 7.0!
```

Você pode ter algumas dúvidas ao ver esse código como por exemplo querer saber o que é uma lista. Vamos discutir esse assunto um pouco mais a frente. Eu apenas quis dar um exemplo. Usar uma lista para dizer quantas vezes queremos repetir um dado conjunto de instruções não é a melhor estratégia pois caso quizessemos imprimir a mensagem do IFBA 100 vezes teríamos que criar uma lista enorme, tipo lista = [1, 2, 3, 4, ..., 100] . Para superar isso podemos usar o comando **range()** . Se apenas um valor positivo for passado dentro da função range, ele irá gerar um intervalo de número iniciando de 0 até um valor a menos do que foi passado. Por exemplo, se o valor 50 for passado, serão gerados os números de 0 até 49 incrementando de um em um como mostra o código abaixo.

```
[5]: for numero in range(50):  
      print("0 ifba é nota 7.0!")
```

```
0 ifba é nota 7.0!  
0 ifba é nota 7.0!
```

[illegible]

Você poderia ficar na dúvida se realmente foram impressos 50 vezes a mensagem. Para isso, é possível usar junto com a mensagem a variável que recebe esses valores contidos no intervalo. A variável x por exemplo, a cada instante receberá um dos valores contidos no intervalo do range.

```
[8]: for x in range(50):  
      print(x, " O ifba é nota 7.0!")  
      print("Mas pode melhorar..")
```

```
0  O ifba é nota 7.0!  
1  O ifba é nota 7.0!  
2  O ifba é nota 7.0!  
3  O ifba é nota 7.0!  
4  O ifba é nota 7.0!  
5  O ifba é nota 7.0!  
6  O ifba é nota 7.0!  
7  O ifba é nota 7.0!  
8  O ifba é nota 7.0!  
9  O ifba é nota 7.0!  
10 O ifba é nota 7.0!  
11 O ifba é nota 7.0!  
12 O ifba é nota 7.0!  
13 O ifba é nota 7.0!  
14 O ifba é nota 7.0!  
15 O ifba é nota 7.0!  
16 O ifba é nota 7.0!  
17 O ifba é nota 7.0!  
18 O ifba é nota 7.0!  
19 O ifba é nota 7.0!  
20 O ifba é nota 7.0!  
21 O ifba é nota 7.0!  
22 O ifba é nota 7.0!  
23 O ifba é nota 7.0!  
24 O ifba é nota 7.0!  
25 O ifba é nota 7.0!  
26 O ifba é nota 7.0!  
27 O ifba é nota 7.0!  
28 O ifba é nota 7.0!  
29 O ifba é nota 7.0!  
30 O ifba é nota 7.0!  
31 O ifba é nota 7.0!  
32 O ifba é nota 7.0!  
33 O ifba é nota 7.0!  
34 O ifba é nota 7.0!  
35 O ifba é nota 7.0!  
36 O ifba é nota 7.0!  
37 O ifba é nota 7.0!  
38 O ifba é nota 7.0!  
39 O ifba é nota 7.0!
```

```
40 0 ifba é nota 7.0!
41 0 ifba é nota 7.0!
42 0 ifba é nota 7.0!
43 0 ifba é nota 7.0!
44 0 ifba é nota 7.0!
45 0 ifba é nota 7.0!
46 0 ifba é nota 7.0!
47 0 ifba é nota 7.0!
48 0 ifba é nota 7.0!
49 0 ifba é nota 7.0!
Mas pode melhorar..
```

É possível passar mais de um valor para a função `range()`. Caso dois valores sejam passados, o primeiro funciona como valor inicial e o segundo como valor final. O incremento, continua o padrão que é de um em um.

```
[14]: for x in range(40, 50):
      print(x, " 0 ifba é nota 7.0!")
```

```
40 0 ifba é nota 7.0!
41 0 ifba é nota 7.0!
42 0 ifba é nota 7.0!
43 0 ifba é nota 7.0!
44 0 ifba é nota 7.0!
45 0 ifba é nota 7.0!
46 0 ifba é nota 7.0!
47 0 ifba é nota 7.0!
48 0 ifba é nota 7.0!
49 0 ifba é nota 7.0!
```

Caso você queira alterar o incremento, é possível passar um terceiro valor que será responsável por incrementar o valor inicial. Nesse caso abaixo o valor de incremento é 5. Isso significa que o intervalo como em 1, o comando `print` será executado em seguida, depois o valor inicial é incrementado em 5, o que resulta em 6, como 6 é menor que 10, o `print` é executado novamente. Em seguida o valor 6 é incrementado de 5, o que resulta em 11, como 11 é maior que o limite colocado que no caso foi 10, o loop se encerra.

```
[13]: for x in range(1, 10, 5):
      print(x, " 0 ifba é nota 7.0!")
```

```
1 0 ifba é nota 7.0!
6 0 ifba é nota 7.0!
```

É possível gerarmos uma sequência no sentido inverso, desde que o incremento seja negativo.

```
[17]: for x in range(10, 0, -1):
      print(x, " 0 ifba é nota 7.0!")
```

E o que acontece se por acaso você configura o `range` de um modo errado, como por exemplo assim: `range(10, 0, 1)`? Se você ainda não percebeu o erro é o seguinte. Essa configuração está dizendo que

o valor inicial será 10 e que iremos em direção ao 0, porém incrementando de um em um. Acontece que se fizermos isso nunca iremos alcançar o 0 pois ao somar 1 com 10 iremos para 11, depois 12, 13, 14 e assim por diante. Isso é um erro. Na programação esse fenômeno é chamado de loop infinito. Só uma observação, existem programas e programas. Tem programas que fazem sentido serem executados em loop infinitos, mas não seria desse modo que estamos discutindo. Abaixo está um exemplo de configuração errada do range e o interpretador do python ao perceber isso, nem permite que seja executado o loop.

```
[1]: for x in range(10, 0, 1):  
      print(x, " O ifba é nota 7.0!")  
      print("Fim do loop")
```

Fim do loop